

Lecture 2

In Class Exercises



Exercise 1

- Consider the SUM-ARRAY procedure below:

```
SUM-ARRAY( $A, n$ )  
1   $sum = 0$   
2  for  $i = 1$  to  $n$   
3       $sum = sum + A[i]$   
4  return  $sum$ 
```

- State a loop invariant for this procedure, and use its initialization, maintenance, and termination properties to show that it returns the sum of the numbers in $A[1:n]$.

Exercise 1 (solution)

Loop invariant: At the start of each iteration i of the **for** loop, $sum = A[1] + A[2] + \dots + A[i - 1]$.

Now, we will use the initialization, maintenance, and termination properties of the loop invariant to show that SUM-ARRAY returns the sum of the numbers in $A[1 : n]$.

Initialization: Upon entering the first iteration, $i = 1$. The sum $A[1] + A[2] + \dots + A[i - 1]$ is empty, since $i - 1 = 0$. The sum of no terms is the identity 0 for addition.

Maintenance: Assume that the loop invariant is true, so that upon entering an iteration for a value of i , $sum = A[1] + A[2] + \dots + A[i - 1]$. The iteration adds $A[i]$ into sum and then increments i , so that the loop invariant holds entering the next iteration.

Termination: The loop terminates once $i = n + 1$. According to the loop invariant, $sum = A[1] + A[2] + \dots + A[n]$. Therefore, sum , which the procedure returns, is indeed the sum of $A[1 : n]$.

Exercise 2

- ❑ Consider sorting n numbers stored in array $A[1:n]$ by first finding the smallest element in $A[1:n]$ and exchanging it with the element in $A[1]$. Then find the smallest element of $A[2:n]$, and exchange it with $A[2]$, then find the smallest element of $A[3:n]$ and exchange it with $A[3]$. Continue in this manner for the first $n-1$ elements of A .
- ❑ Write the pseudocode for this algorithm.
- ❑ What loop invariant does this algorithm maintain?
- ❑ Why does it need to run for the first $n-1$ elements, rather than all n elements?
- ❑ Give the worst-case running time in Θ -notation.
- ❑ Is the best-case running time any better?

Exercise 2 (solution)

SELECTION-SORT(A, n)

for $i = 1$ **to** $n - 1$

$smallest = i$

for $j = i + 1$ **to** n

if $A[j] < A[smallest]$

$smallest = j$

 exchange $A[i]$ with $A[smallest]$

Exercise 2 (solution)

- The algorithm maintains the loop invariant that at the start of each iteration of the outer **for** loop the subarray $A[1:i-1]$ consists of the $i-1$ smallest elements in the array $A[1:n]$, and this subarray is in sorted order.
- After the first $n-1$ elements, the subarray $A[1:n-1]$ contains the smallest $n-1$ elements, sorted, and therefore element $A[n]$ must be the largest element.
- The running time of the algorithm is $\Theta(n^2)$ for all cases.

Exercise 3

- ❑ Input: A sequence of n numbers $\{a_1, a_1, \dots, a_n\}$ stored in array $A[1:n]$, a range [low:high] to search for and a value x .
- ❑ Output: An index i such that x equals $A[i]$ or the special value *NIL* if x does not appear in A .
- ❑ Assume that the array A is already sorted.
- ❑ Write the **binary search** algorithm which searches for x by halving the size of the remaining portion of the subarray each time. Write pseudocode, either iterative or recursive.
- ❑ Argue that the worst-case running time for binary search is $\Theta(\lg n)$.

Exercise 3 (solution)

ITERATIVE-BINARY-SEARCH($A, x, low, high$)

while $low \leq high$

$mid = \lfloor (low + high) / 2 \rfloor$

if $x == A[mid]$

return mid

elseif $x > A[mid]$

$low = mid + 1$

else $high = mid - 1$

return NIL

Exercise 3 (solution)

RECURSIVE-BINARY-SEARCH($A, x, low, high$)

if $low > high$

return NIL

$mid = \lfloor (low + high) / 2 \rfloor$

if $x == A[mid]$

return mid

elseif $x > A[mid]$

return RECURSIVE-BINARY-SEARCH($A, x, mid + 1, high$)

else return RECURSIVE-BINARY-SEARCH($A, x, low, mid - 1$)

Exercise 3 (solution)

- Both procedures terminate the search unsuccessfully when the range is empty (i.e., $low > high$) and terminate it successfully if the value x has been found.
- Based on the comparison of x to the middle element in the searched range, the search continues with the range halved.
- The recurrence for these procedures is therefore
$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1) = \Theta(\lg n).$$